Package 'golemcrypto'

October 31, 2026

Title Interactive Cryptocurrency Time Series and Market Data Explorer

Version 0.1.0

Maintainer Dimitrios Zacharatos <dzacharatos@yahoo.com>

Description A 'shiny' application for exploring cryptocurrency exchange rates. Plots interactive candlestick and volume charts of historical prices since 2013 for any active cryptocurrency, and shows a table of the top 100 cryptocurrencies by market capitalisation with price changes over 24 hours to 90 days, market dominance and fully diluted market cap. Also compares normalized prices of several coins, and shows a market cap treemap and a calendar of monthly returns. Market data is retrieved from 'CoinMarketCap'.

Imports bslib,
cachem,
coinmarketcapr,
crypto2,
curl,
DT,
jsonlite,
plotly,
shiny,
shinycssloaders,
shinyWidgets,
stats,
tools,
utils

Suggests htmltools,
testthat (>= 3.0.0)

Config/testthat/edition 3

License GPL-3

URL https://sedzinfo.github.io/golemcrypto/, https://github.com/sedzinfo/golemcrypto

BugReports https://github.com/sedzinfo/golemcrypto/issues

Encoding UTF-8

Roxygen list(markdown = TRUE)

Config/roxygen2/version 8.1.1

Contents

crypto 2
drawdown 3
get_coins 4
get_history 5
get_listings 6
listings_table 6
monthly_returns 7
moving_average 8
normalize_prices 8
plot_compare 9
plot_returns_calendar 10
plot_timeseries 10
plot_treemap 11

Index **13**

`crypto` *Launch the Crypto Time Series Analyzer*

Description

Starts an interactive 'shiny' application for exploring cryptocurrency exchange rates and market data from 'CoinMarketCap'.

Usage

```
crypto(
  api_key = Sys.getenv("COINMARKETCAP_APIKEY"),
  analytics = FALSE,
  keep_alive = TRUE,
  ...
)
```

Arguments

`api_key`	A 'CoinMarketCap' API key, needed for the market listings. By default it is read from the `COINMARKETCAP_APIKEY` environment variable, which can be set in `.Renviron`.
`analytics`	`TRUE` to add Google Analytics, through the cookie banner shared with the sites of sedzinfo.github.io, which loads it only after the visitor accepts. Meant for the public deployment (see `app.R`); off by default, so that running the app yourself sends nothing.

keep_alive	TRUE (the default) to ping the server every 15 seconds while the page is open, so that proxies do not close idle connections. FALSE lets the app go to sleep when nobody uses it, as app.R does on shinyapps.io, where an app that stays awake uses up the active hours of the plan.
...	Further arguments passed to shiny::shinyApp(), for example options = list(port = 8080).

Details

The application has the following tabs:

Timeseries Candlestick chart of daily open, high, low and close prices in USD since 2013 for any active cryptocurrency, with its 50- and 200-day moving averages, and charts of the volume and of the drawdown (how far the price is below its all-time high) underneath.

Top 100 Table of the top 100 cryptocurrencies by market cap with price, volume, 24h to 90d price changes, market dominance and fully diluted market cap.

Compare Close prices of up to 10 coins, normalized to 100 on the first date all of them have data.

Treemap Top 100 coins sized by market cap and colored by 24h change.

Returns calendar Monthly returns by year for the coin selected in the Timeseries tab.

Market listings are downloaded when the app starts and refreshed every 10 minutes. When they cannot be downloaded, for example because the API key has used up its credits, the app still starts, and the Top 100 and Treemap tabs tell the visitors why and when the limit resets. Historical prices are downloaded on demand with crypto2::crypto_history() and kept on disk between R sessions, so after the first download only the days since the last one are fetched. An internet connection is required. When 'CoinMarketCap' limits the requests, crypto2 prints "Retrying in 60 seconds" and the chart appears about a minute later.

Value

A shiny::shinyApp() object. Printing it, or calling crypto() at the console, opens the app in the browser.

Examples

```
if (interactive()) crypto()

# with the API key passed directly, on a fixed port
if (interactive()) crypto(api_key = "your-api-key", options = list(port = 8080))
```

---

drawdown *Drawdown of prices*

Description

How far each close is below the highest close up to that day, the all-time high so far, in percent: 0 on a new high and -50 at half of it.

Usage

```
drawdown(close)
```

Arguments

close	Numeric vector of close prices, sorted by date.

Value

A numeric vector as long as close, from -100 to 0.

Examples

```
drawdown(c(100, 120, 90, 130, 65))
```

---

get_coins *List the active coins*

Description

Downloads the list of the active coins tracked by 'CoinMarketCap' and keeps it on disk for a day (see disk_cache()). When the download fails, the kept list is used. Its name_symbol column holds the labels, such as "Bitcoin (BTC)", that pick coins in get_history().

Usage

```
get_coins()
```

Value

The crypto2::crypto_list() data frame, with a name_symbol column.

Examples

```
## Not run:
coins <- get_coins()
head(coins$name_symbol)

## End(Not run)
```

get_history *Get the daily price history of coins*

Description

Returns the daily USD prices of one or more coins from 'CoinMarketCap'. Each coin's history is kept on disk between R sessions, and once it is more than an hour old only the days since its last day are downloaded, so calls are fast and stay below the request limit of 'CoinMarketCap'. When that limit is hit, crypto2 waits a minute and tries again.

Usage

```
get_history(labels, start_date = Sys.Date() - 365, coins = get_coins())
```

Arguments

labels	Character vector of coin labels, such as "Bitcoin (BTC)", from the name_symbol column of get_coins().
start_date	Date. The first day to get, a year ago by default.
coins	Data frame of coins from get_coins(), to look the labels up.

Value

A data frame with one row per coin and day, sorted by coin and date, with the open, high, low, close and volume columns, timestamp as a Date and the name_symbol label. Labels that match no coin are left out with a message, and NULL is returned when no label matches or nothing could be downloaded.

Examples

```
## Not run:
btc <- get_history("Bitcoin (BTC)", as.Date("2020-01-01"))
plot_timeseries(btc)

history <- get_history(c("Bitcoin (BTC)", "Ethereum (ETH)"))
plot_compare(normalize_prices(history))

## End(Not run)
```

get_listings *Get the latest market listings*

Description

Returns the listings of the coins by market cap, with their price, volume, price changes from 24 hours to 90 days, market cap and dominance in USD. The listings are downloaded again when they are older than 10 minutes, and the download is shared by all sessions and app runs in the same R process, so the API is called at most once in that time.

Usage

```
get_listings()
```

Details

The listings need a 'CoinMarketCap' API key in the `COINMARKETCAP_APIKEY` environment variable, see the README. A failed download, for example when the key has used up its credits, is tried again after 10 minutes. Until then the previous listings and their time are kept, so the time stays truthful, and `problem` tells the visitors of the app why, see `listings_problem()`.

Value

A list with `data`, a data frame with one row per coin sorted by market cap, and `time`, when it was downloaded (both `NULL` until a download works), and `problem`, `NULL` when the last download worked and otherwise a sentence on why it failed.

Examples

```
## Not run:
listings <- get_listings()
listings$time
head(listings$data)

## End(Not run)
```

listings_table *Table of the market listings*

Description

A sortable table with 100 rows per page and numbers rounded to 2 decimals, showing the price in dollars and the price changes in percent. Losses are shaded red and gains green, more strongly the larger the change, in steps of 0.1 percentile from the 5th to the 95th. Changes near zero keep the table's own background, so the table suits the light and the dark mode.

Usage

```
listings_table(listings)
```

Arguments

listings	The data of get_listings().

Value

A DT datatable.

Examples

```
## Not run:
listings_table(get_listings()$data)

## End(Not run)
```

---

monthly_returns *Monthly returns of a coin*

Description

Each month's return is its last close relative to the last close of the month before, in percent. The first month, and the months still to come, are NA.

Usage

```
monthly_returns(history)
```

Arguments

history	Price history of one coin from get_history(), with timestamp and close columns, sorted by date.

Value

A year by month matrix of returns in percent, rounded to 2 decimals, with the years as row names and month.abb as column names.

Examples

```
history <- data.frame(
  timestamp = as.Date(c("2024-01-31", "2024-02-29", "2024-03-31")),
  close = c(100, 110, 121)
)
monthly_returns(history)
```

moving_average *Moving average of prices*

Description

The simple moving average: each day's value is the mean of the last `days` closes, that day included, so the first `days - 1` days have none.

Usage

```
moving_average(close, days)
```

Arguments

`close`	Numeric vector of close prices, sorted by date.
`days`	Number of days to average, such as 50 or 200.

Value

A numeric vector as long as `close`, `NA` for the first `days - 1` days, and all `NA` when there are fewer than `days` closes, as for a coin younger than the average.

Examples

```
moving_average(c(1, 2, 3, 4, 5), days = 3)
```

normalize_prices *Normalize close prices to 100*

Description

Every coin starts at 100 on the first date on which all of them have data, so their lines can be compared directly: 150 means the price rose by half since that date.

Usage

```
normalize_prices(history)
```

Arguments

`history`	Price history from `get_history()`, with `name_symbol`, `timestamp` and `close` columns, sorted by coin and date.

Value

`history` from the first common date on, with an `index` column.

Examples

```
history <- data.frame(
  name_symbol = rep(c("Alpha (A)", "Beta (B)"), each = 3),
  timestamp = rep(as.Date("2024-01-01") + 0:2, 2),
  close = c(10, 11, 12, 200, 180, 220)
)
normalize_prices(history)
```

`plot_compare` *Line chart of normalized close prices*

Description

One line per coin, with buttons that switch the y axis between a linear and a log scale.

Usage

```
plot_compare(prices, mode = "light")
```

Arguments

`prices`	Prices from `normalize_prices()`.
`mode`	`"light"` (the default) or `"dark"`, to suit a light or dark page.

Value

A plotly object.

Examples

```
## Not run:
history <- get_history(c("Bitcoin (BTC)", "Ethereum (ETH)", "Solana (SOL)"))
plot_compare(normalize_prices(history))

## End(Not run)
```

plot_returns_calendar *Heatmap of monthly returns by year*

Description

Months are colored from red to green by their return. The color limit is the 95th percentile of the absolute returns, so a few extreme months do not wash out the rest.

Usage

```
plot_returns_calendar(calendar, label = NULL, mode = "light")
```

Arguments

calendar	Matrix from monthly_returns().
label	Label of the coin for the title, or NULL for none.
mode	"light" (the default) or "dark", to suit a light or dark page.

Value

A plotly object.

Examples

```
## Not run:
btc <- get_history("Bitcoin (BTC)", as.Date("2013-04-29"))
plot_returns_calendar(monthly_returns(btc), "Bitcoin (BTC)")

## End(Not run)
```

plot_timeseries *Candlestick chart of a coin's prices*

Description

Plots the daily open, high, low and close prices, with a line through the close, its moving averages, date range buttons and a range slider. Underneath, smaller charts show the trading volume and the drawdown: how far the price is below its all-time high so far. A click on a legend entry hides or shows that line.

Usage

```
plot_timeseries(
  history,
  mode = "light",
  averages = c(50, 200),
  show_drawdown = TRUE
)
```

Arguments

history	Price history of one coin from get_history(). The averages and the drawdown use the whole history, so pass all of it and zoom with the range buttons.
mode	"light" (the default) or "dark", to suit a light or dark page.
averages	Days of the moving averages to draw (see moving_average()), or NULL for none.
show_drawdown	TRUE to add the drawdown chart (see drawdown()).

Value

A plotly object.

Examples

```
## Not run:
btc <- get_history("Bitcoin (BTC)", as.Date("2013-04-29"))
plot_timeseries(btc)

# shorter averages, and no drawdown chart
plot_timeseries(btc, averages = c(20, 100), show_drawdown = FALSE)

## End(Not run)
```

---

plot_treemap *Treemap of the top 100 coins*

Description

Each box is sized by market cap and colored from red to green by the 24h price change. The color limits are the 5th and 95th percentiles of the changes, so a few outliers do not wash out the rest. Coins without a 24h change are labelled n/a and colored white.

Usage

```
plot_treemap(listings, mode = "light")
```

Arguments

listings	The data of get_listings().
mode	"light" (the default) or "dark", to suit a light or dark page.

Value

A plotly object.

Examples

```
## Not run:
plot_treemap(get_listings()$data)

## End(Not run)
```

Index

∗ **crypto**
    crypto, 2
∗ **timeseries**
    crypto, 2

crypto, 2
crypto2::crypto_history(), *3*
crypto2::crypto_list(), *4*

disk_cache(), *4*
drawdown, 3
drawdown(), *11*

get_coins, 4
get_coins(), *5*
get_history, 5
get_history(), *4*, *7*, *8*, *11*
get_listings, 6
get_listings(), *7*, *11*

listings_problem(), *6*
listings_table, 6

monthly_returns, 7
monthly_returns(), *10*
moving_average, 8
moving_average(), *11*

normalize_prices, 8
normalize_prices(), *9*

plot_compare, 9
plot_returns_calendar, 10
plot_timeseries, 10
plot_treemap, 11

shiny::shinyApp(), *3*